

EE5207 Advanced Computer Architecture

Semester 2/ 2025-2026

Design of a Vending Machine

Student: Nguyen Minh Duc - 2570662

1. Introduction

Vending Machine is a dispenser machine that receives coins or bills and dispenses soft drinks or snacks. In this project, our vending machine will receive coins with 3 values – nickel (5¢), dime (10¢) and quarter (25¢), then the vending machine will dispense a soda can that costs (20¢).



Figure 1: Vending machine

2. Design Strategy

2.1. Overview

The coding style is structural + behavioral using SystemVerilog. No math signs (+, -, *, /) are used, instead, we build adder and subtractor modules from scratch using half adder, full adder, etc. Noticed that we use gate logic inside modules to perform the structural design.

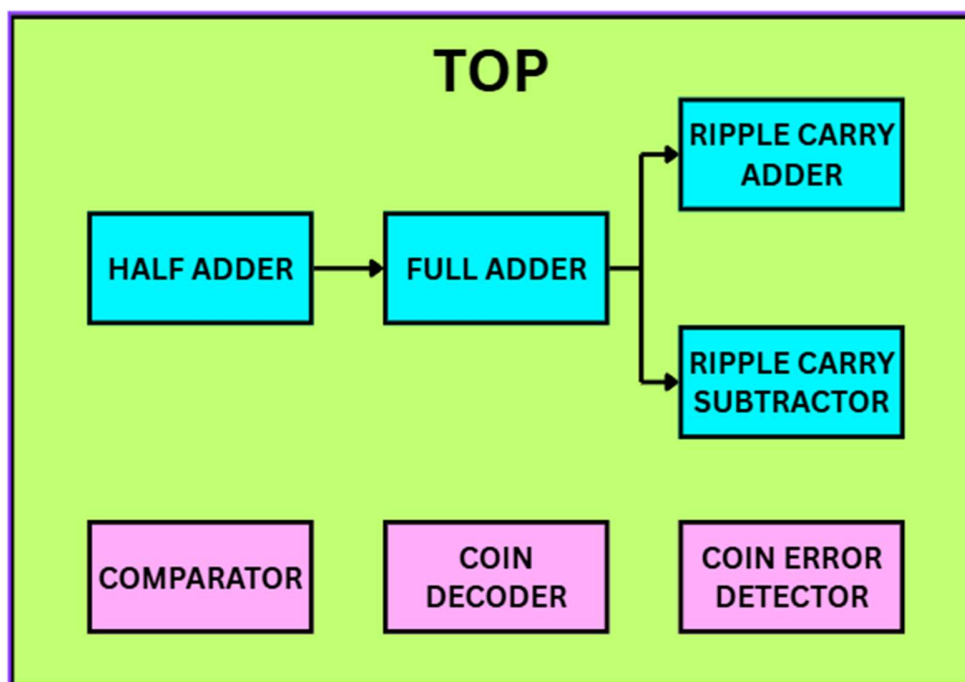


Figure 2.1. Modules Design Strategy

2.2. Half Adder

A half adder is a basic combinational circuit that adds two single-bit binary inputs (A and B) to produce a SUM using an XOR gate and a CARRY using an AND gate, without considering any carry-in from a previous stage.

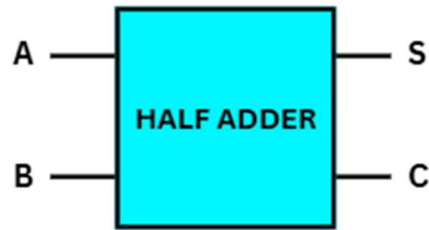


Figure 2.2. Half Adder Block Diagram

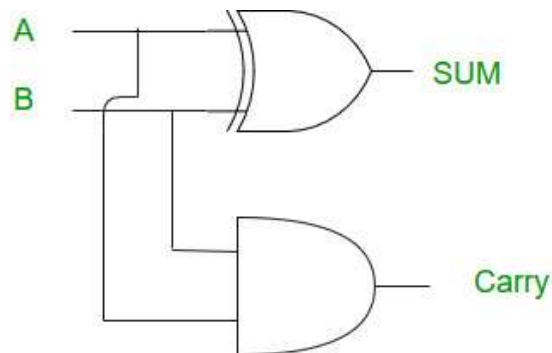


Figure 2.3. Half Adder Circuit Diagram

Table 2.1. Half Adder Truth Table

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

2.2.1. Specification

Signal	Width	Direction	Description
A	1	Input	First addend.
B	1	Input	Second addend.
S	1	Output	Sum.
C	1	Output	Carry.

2.3. Full Adder

Full Adder is a combinational circuit that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C-IN. The output carry is designated as C-OUT and the normal output is designated as S which is SUM.

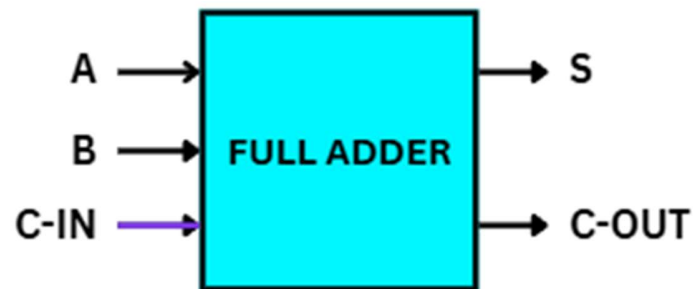


Figure 2.4. Full Adder Block Diagram

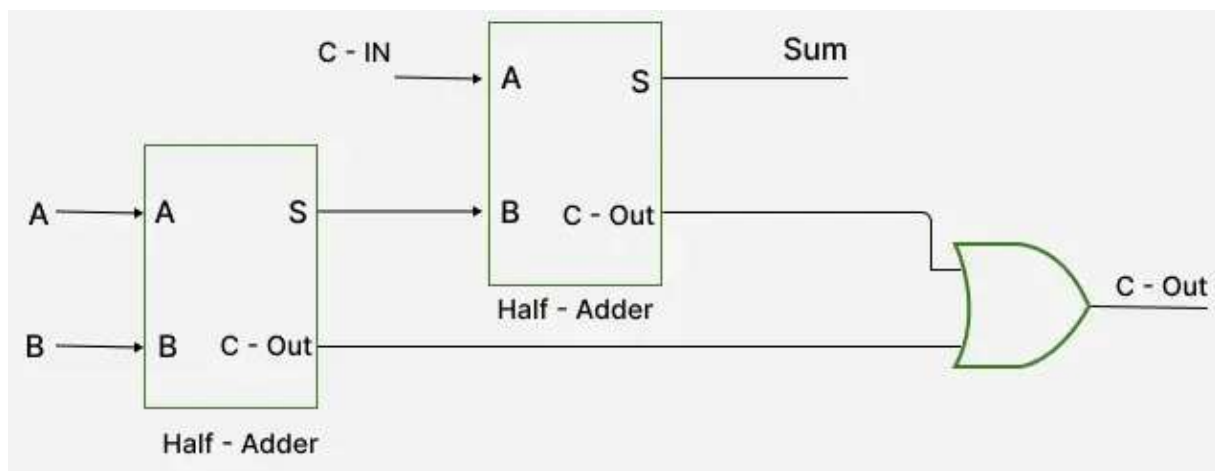


Figure 2.5. Full Adder Circuit Diagram

Table 2.2. Full Adder Truth Table

INPUT			OUTPUT	
A	B	C-IN	S	C-OUT
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

2.3.1. Specification

Signal	Width	Direction	Description
A	1	Input	First addend.
B	1	Input	Second addend.
C-IN	1	Input	Carry-in
S	1	Output	Sum.
C-OUT	1	Output	Carry-out.

2.4. Ripple Carry Adder

In ripple carry adders, for each adder block, the two bits that are to be added are available instantly. However, each adder block waits for the carry to arrive from its previous block. So, it is not possible to generate the sum and carry of any block until the input carry is known. The i^{th} block waits for the $(i - 1)^{th}$ block to produce its carry.

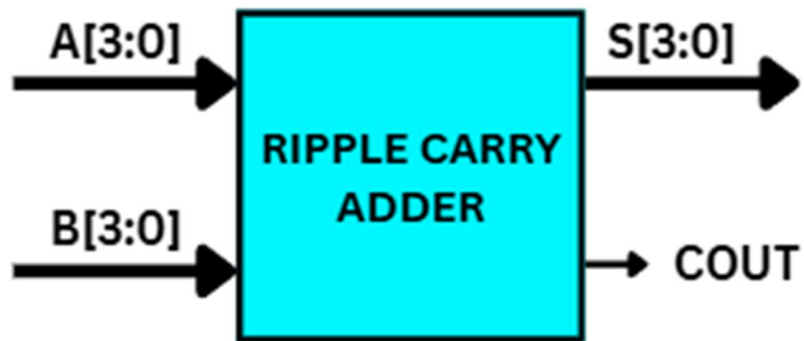


Figure 2.6. Ripple Carry Adder Block Diagram

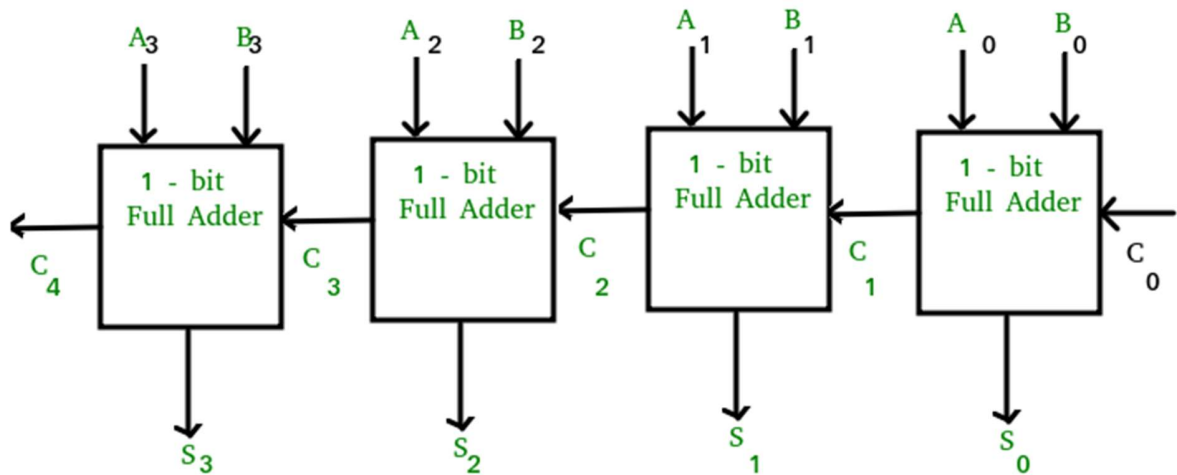


Figure 2.7. Ripple Carry Adder Circuit Diagram

2.4.1. Specification

Signal	Width	Direction	Description
A	4	Input	First 4-bit addend.
B	4	Input	Second 4-bit addend.
S	4	Output	4-bit Sum.
C-OUT	1	Output	Carry-out.

2.5. Ripple Carry Subtractor

A ripple carry subtractor is a digital circuit that subtracts two N -bit binary number by cascading N full adders. It performs subtraction ($A - B$) by taking two's complement of subtrahend (B), adding it to the minuend (A), and setting the initial carry to 1, which acts as the +1 in the two's complement process.

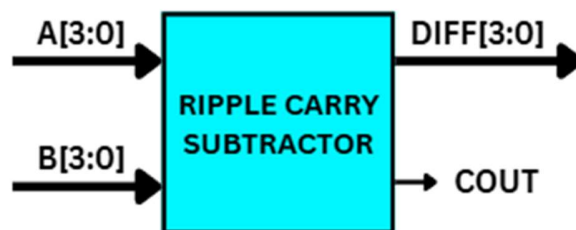


Figure 2.8. Ripple Carry Subtractor Block Diagram

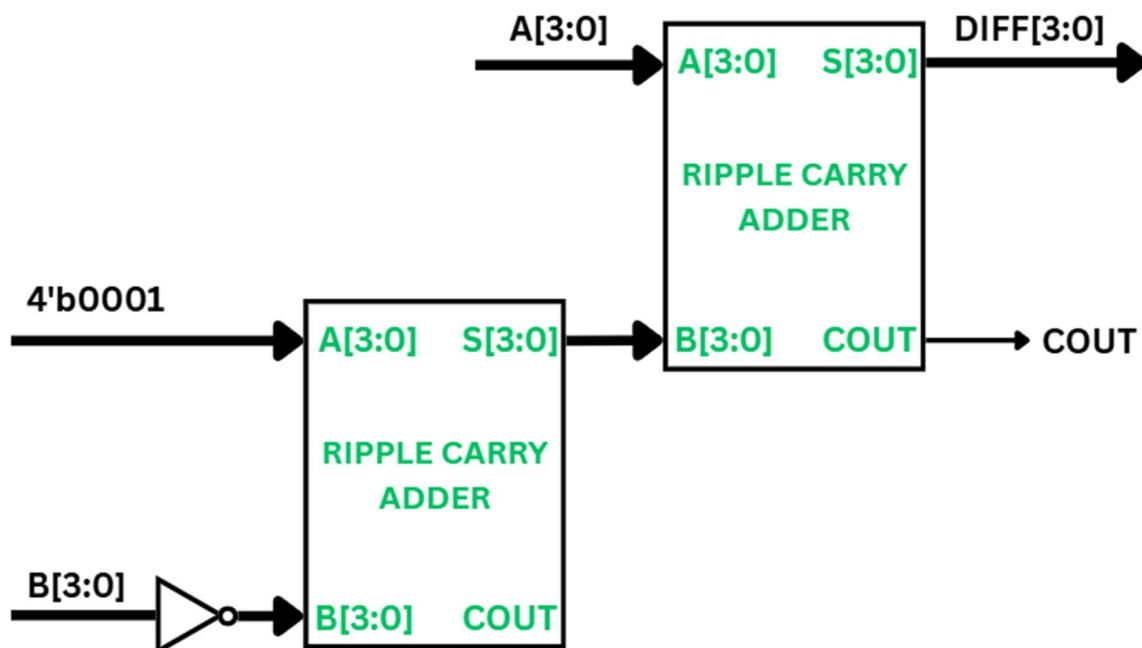


Figure 2.9. Ripple Carry Subtractor Circuit Diagram

2.5.1. Specification

Signal	Width	Direction	Description
A	4	Input	First 4-bit addend.
B	4	Input	Second 4-bit addend.
DIFF	4	Output	4-bit Difference.
C-OUT	1	Output	Carry-out.

The ripple carry subtractor is used to calculate the change when the inserted coin exceeds the soda price through the *o_change* output signal. The *o_change* signal is a 3-bit output, where each value represents a specific amount of returned change.

Table 2.3. o_change signal encoding

o_change[2:0]	C-OUT
000	The change is 0¢
001	The change is 5¢
010	The change is 10¢
011	The change is 15¢
100	The change is 20¢
101	Reserved
110	Reserved
111	Reserved

The *o_soda* signal is asserted when the value of *o_credit* is greater than or equal to 20¢, indicating that a soda should be dispensed. If the accumulated credit exceeds the soda price, the extra amount is returned as change through the *o_change* output, which is expected to be $o_change = o_credit - 20¢$.

The encoding of the *o_credit* signal is defined in Table 2.4.

Table 2.4. *o_credit* signal encoding

o_change[3:0]	C-OUT
0000	The credit is 0¢
0001	The credit is 5¢
0010	The credit is 10¢
0011	The credit is 15¢
0100	The credit is 20¢
0101	The credit is 25¢
0110	The credit is 30¢
0111	The credit is 35¢
1000	The credit is 40¢
1001	Reserved
...	Reserved
1111	Reserved

2.6. Comparator

The comparator is specified for checking if the accumulated credit *o_credit* is equal or greater than 20¢ (4'b0100). The output signal is used for dispense the soda.



Figure 2.10. Comparator Block Diagram

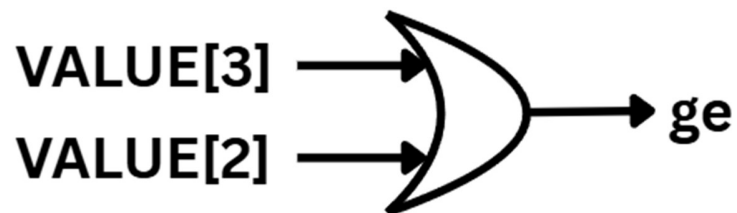


Figure 2.11. Comparator Circuit Diagram

Table 2.2. Full Adder Truth Table

VALUE[3]	VALUE[2]	ge
0	0	0
0	1	1
1	0	1
1	1	1

2.6.1. Specification

Signal	Width	Direction	Description
VALUE	4	Input	Comparing value.
ge	1	Output	Equal or greater valid.

2.7. Coin Decoder

The coin decoder will take the coin input and decode into 4-bit binary value. The output value is used to calculate the total coins value and the change if it exceeds 20¢ (4'b0100).

Table 2.3. Coin Types

Coin	Value	Binary
Nickel	5¢	4'b0001
Dime	10¢	4'b0010
Quarter	25¢	4'b0101

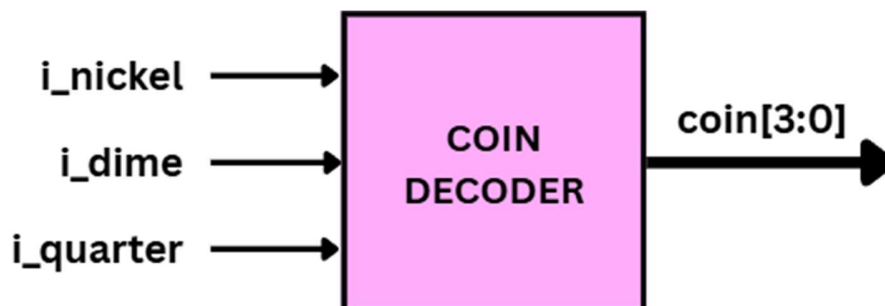


Figure 2.12. Coin Decoder Block Diagram

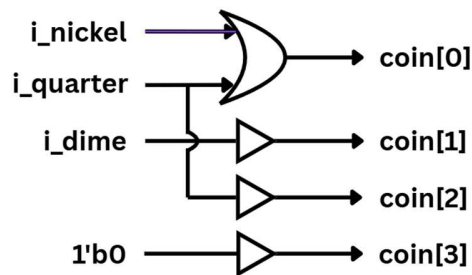


Figure 2.13. Coin Decoder Circuit Diagram

2.7.1. Specification

Signal	Width	Direction	Description
<code>i_nickel</code>	1	Input	Nickel - 5¢.
<code>i_dime</code>	1	Input	Dime - 10¢.
<code>i_quarter</code>	1	Input	Quarter - 25¢.
<code>coin</code>	4	Output	Binary coin type.

2.8. Coin Error Detector

The coin error detector takes the coin input and check if there are more than 2 coins inserted at the same time; therefore, no soda is dispensed. The output value is also the output error of the vending machine.

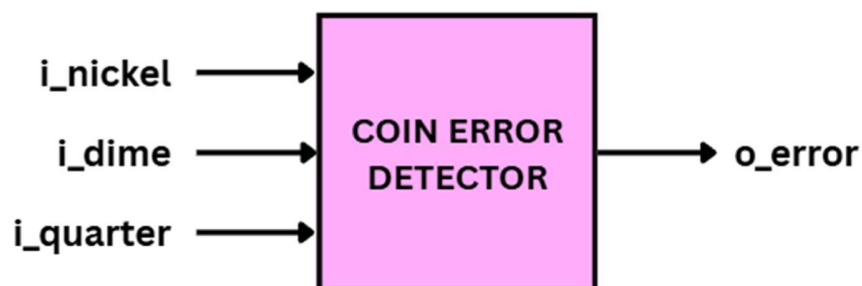


Figure 2.14. Coin Detector Block Diagram

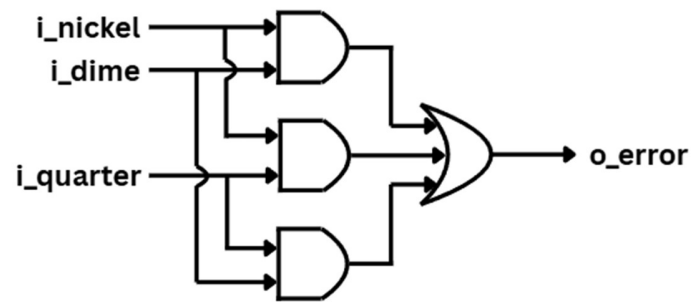


Figure 2.15. Coin Detector Circuit Diagram

2.8.1. Specification

Signal	Width	Direction	Description
<code>i_nickel</code>	1	Input	Nickel - 5¢.
<code>i_dime</code>	1	Input	Dime - 10¢.
<code>i_quarter</code>	1	Input	Quarter - 25¢.
<code>o_error</code>	1	Output	Error check.

2.9. Top Module

The top module connects all the modules above and perform the function of the vending machine.

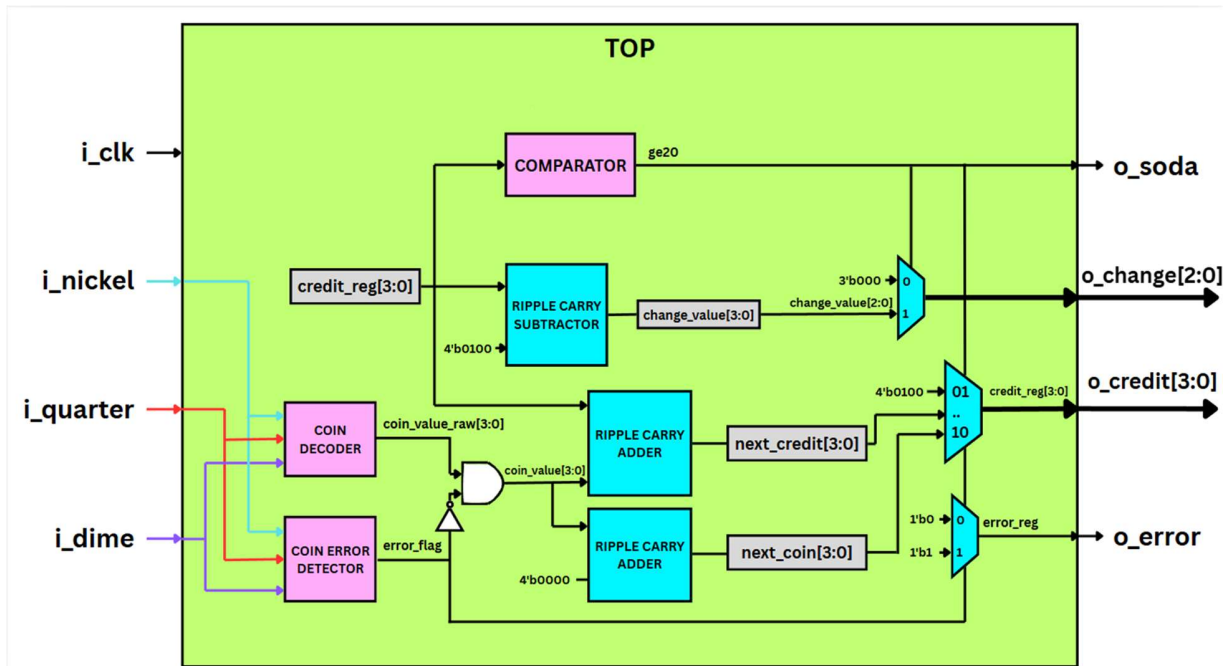


Figure 2.16. Top Module Diagram

2.9.1. Specification

Signal	Width	Direction	Description
<i>i_clk</i>	1	Input	System clock.
<i>i_nickel</i>	1	Input	Nickel - 5¢.
<i>i_dime</i>	1	Input	Dime - 10¢.
<i>i_quarter</i>	1	Input	Quarter - 25¢.
<i>o_soda</i>	1	Output	Soda dispensed signal.
<i>o_change</i>	3	Output	Change when exceeding.
<i>o_credit</i>	4	Output	Accumulated credit.
<i>o_error</i>	1	Output	Error check signal.

3. Result

3.1. Simulation

In **Figure 3.1**, the system receives two nickels followed by a dime from the customer, resulting in a total credit of exactly 20¢. In the subsequent clock cycle, the vending machine dispenses a soda and no change is returned ($o_change = 0¢$).

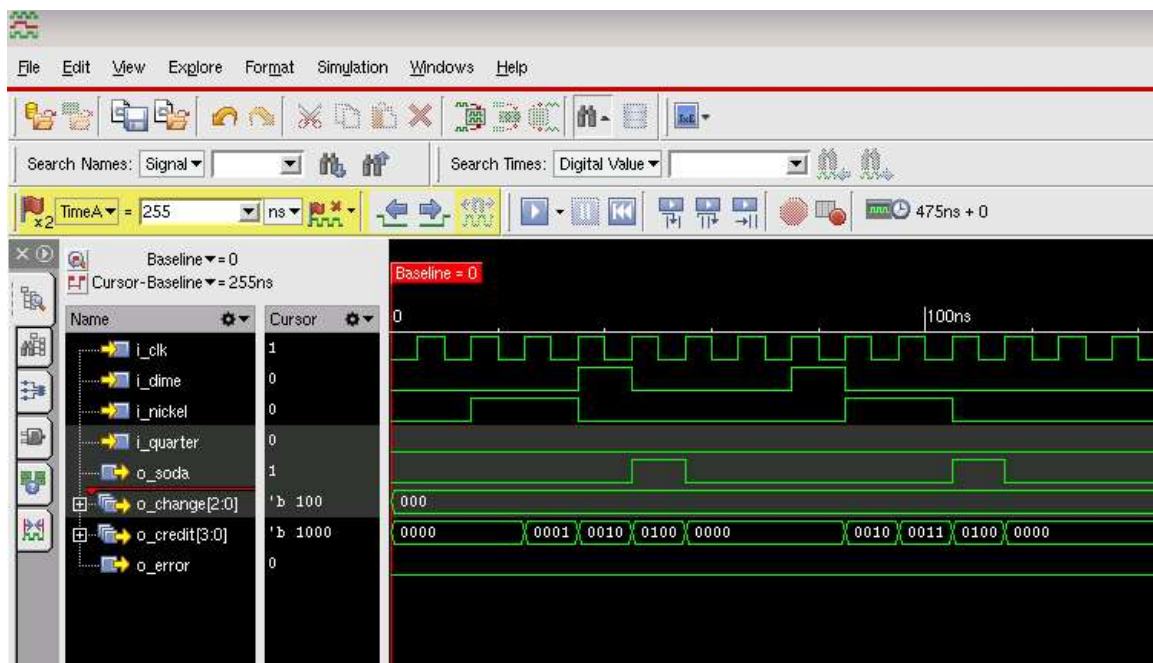


Figure 3.1. Simulation Result

In **Figure 3.2**, the system receives a quarter as input. Since the inserted value exceeds the soda price of 20¢, the machine dispenses a soda in the following clock cycle and returns a change of 5¢. While dispensing the soda, the vending machine may still receive additional coins, which will be registered and processed in the next clock cycle.

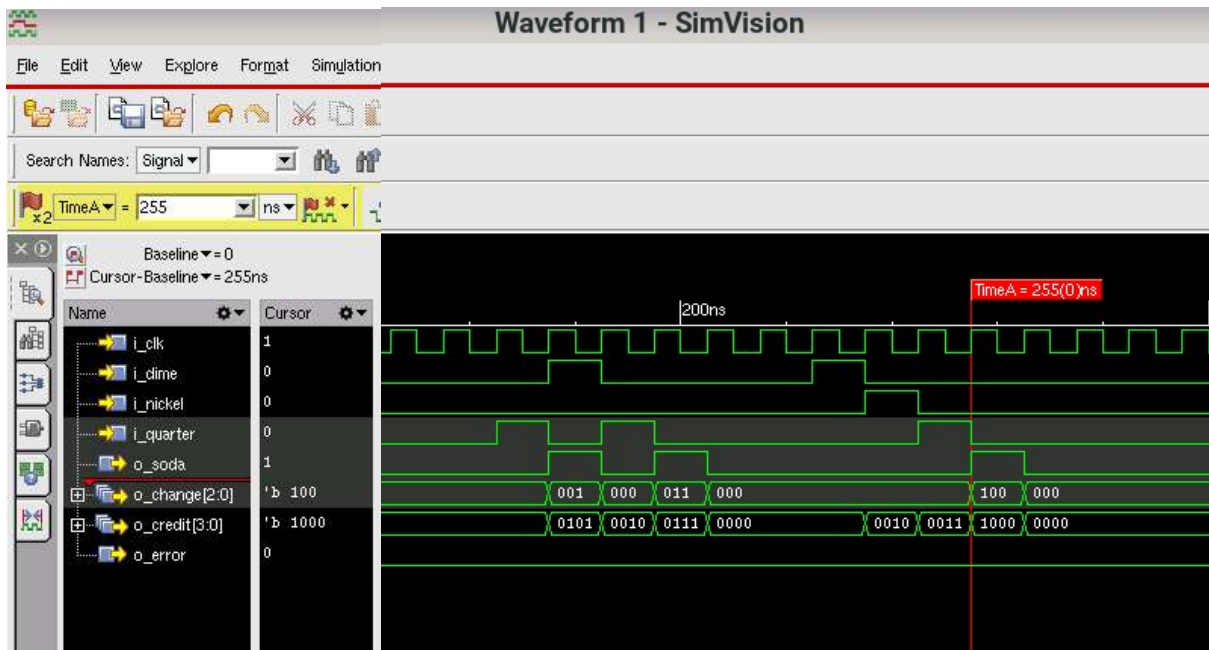


Figure 3.2. Simulation Result

In **Figure 3.3**, the system detects that two or more coin input signals are asserted simultaneously. An error flag is set high to indicate this faulty behavior, the vending machine does not dispense a soda, and both the change and credit outputs are set to 0¢.

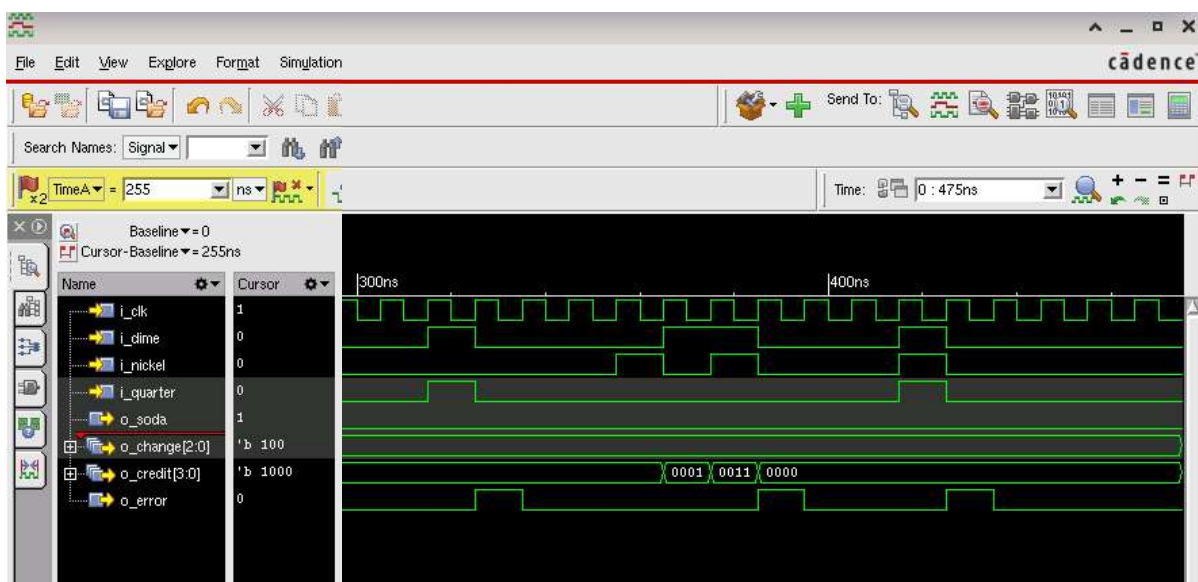


Figure 3.3. Simulation Result